



Welcome to CPSC 210!

If you are enrolled in the course (or are on the waiting list), you will want to sign up for the EdX Edge course. The URL may differ from term-to-term; so, look for a link in Canvas, Piazza, or mail to the course/waitlist from your instructor.

Please do not email the instructors for help registering in the course. If the lecture section in which you wish to enroll is full, please register on the corresponding waitlist. Students will be moved into the lecture sections as seats become available according to the waitlist policy found on the [department's web page](#).

CPSC 210 Course Learning Outcomes	1a. Specify a new program, given a small problem description, by identifying relevant data abstractions, and specifying the associated behaviour of those abstractions. 1b. Validate the program's behaviour against its specification prior to implementation using test driven design, through writing unit tests that achieve basic code coverage. 1c. Implement the program in Java, choosing the internal structure of its abstractions, and carry out implementation choices. 2. Design software using basic Object oriented design principles (coupling, cohesion, single responsibility, substitution), robustness, and design patterns where applicable. 3. Analyse the structure and behaviour of an existing small to medium sized codebase by making static and dynamic models and identifying design idioms, including design patterns, present in the code. 4. Evolve a small to medium sized codebase by changing or augmenting behaviour of the code through alteration, subtyping or composition 5. Recover from common structural issues through identifying those issues, and selecting and applying the appropriate refactorings 6. Communicate (orally through code demonstration) implementation choices, and the rationale for those choices
Topic Type	Quizzes
BASICS	1. Program Structure 2. Methods and Calls 3. Classes, Objects and Variables 4. Data Flow 5. Conditions, Loops, Execution Flow and Debugging
ABSTRACTION	1. Specifying and Using a Data Abstraction 2. Testing a Data Abstraction 3. Implementing a Data Abstraction 4. Types, Substitution, Interfaces 5. Multiple Interfaces 6. Extends, Override, Super 7. Abstract classes, Overloading
CONSTRUCTION	1. Throwing exceptions 2. Unchecked exceptions and exception hierarchies, assertions 3. Testing exceptions 4. Extract design hierarchy, associations 5. Extract sequences 6. Implementing a hierarchy 7. Object Equality: Overriding Hashcode and Equals 8. Reflexive relationships and Sequence Diagrams
DESIGN	1. Coupling and Cohesion 2. Refactoring 3. Liskov Substitution Principle 4. Composite Design Pattern 5. Observer Design Pattern 6. Java Observer Pattern 7. Iterator Pattern 8. Advanced Iterator (Custom iterators)